Aubrée Romain **Beauvallet**
10/11/2026 **Semestre 3**
SLAM

Groupe 1

Bloc 3 TP 1 - Hacking sans installation

Contexte **2**
Les bases de la cybersécurité et du Web **2**
La faille IDOR **5**
La divulgation de fichiers sensibles **7**
L'injection HTML **12**
Le Cross-Site Scripting (XSS) **13**
L’injection SQL **16**
Conclusion **22**

Contexte

Ce TP porte sur le **hacking éthique sans installation**. On n'installe aucun outil ni aucun système : on travaille uniquement avec un **navigateur web** et une connexion Internet.

On aborde trois sujets, dans l'ordre : les **bases** de la sécurité web, la faille **IDOR**, puis la **divulgation de fichiers sensibles**.

Les manipulations se font directement dans le navigateur et, pour les exercices pratiques, sur la plateforme **PortSwigger Web Security Academy**.

Les bases de la cybersécurité et du Web

La triade C.I.A.

La **cybersécurité** protège les systèmes, les réseaux et les **données** contre les attaques et les accès non autorisés.

Elle repose sur trois piliers, appelés la **triade C.I.A.** :

- **Confidentialité** : seules les personnes autorisées peuvent consulter une information privée.

- **Intégrité** : les données ne doivent pas être modifiées sans autorisation.

- **Disponibilité** : les services et les données restent accessibles quand on en a besoin.

Chaque faille web qu'on étudie ensuite cherche à casser l'un de ces trois piliers.

Le fonctionnement du Web et l'anatomie d'une URL

Le Web repose sur le modèle **client-serveur** : le **client** (notre navigateur) envoie une **requête**, le **serveur** traite la demande et renvoie une **réponse** (une page HTML).

Pour atteindre le bon serveur, le navigateur utilise une **URL**, un peu comme une adresse postale unique.

Une URL complète se décompose en plusieurs éléments : le **schéma** (par exemple https), le **sous-domaine** et le **domaine**, le **domaine de premier niveau** (TLD), le **port**, le **chemin** (path) et la **query string** après le point d'interrogation.

En sécurité, c'est surtout la **query string** qui nous intéresse : ses paramètres se modifient à la main dans la barre d'adresse.

Exercice 1 : Décomposition d'une URL

On analyse l'URL **https://www.example.co.uk:4443/blog/article/search?color=blue&sort=newest**. On isole chaque composant :

- Schéma (protocole) : **https**

- Sous-domaine : **www** — domaine principal : **example**

- Domaine de premier niveau (TLD) : **.co.uk**

- Port : **4443**

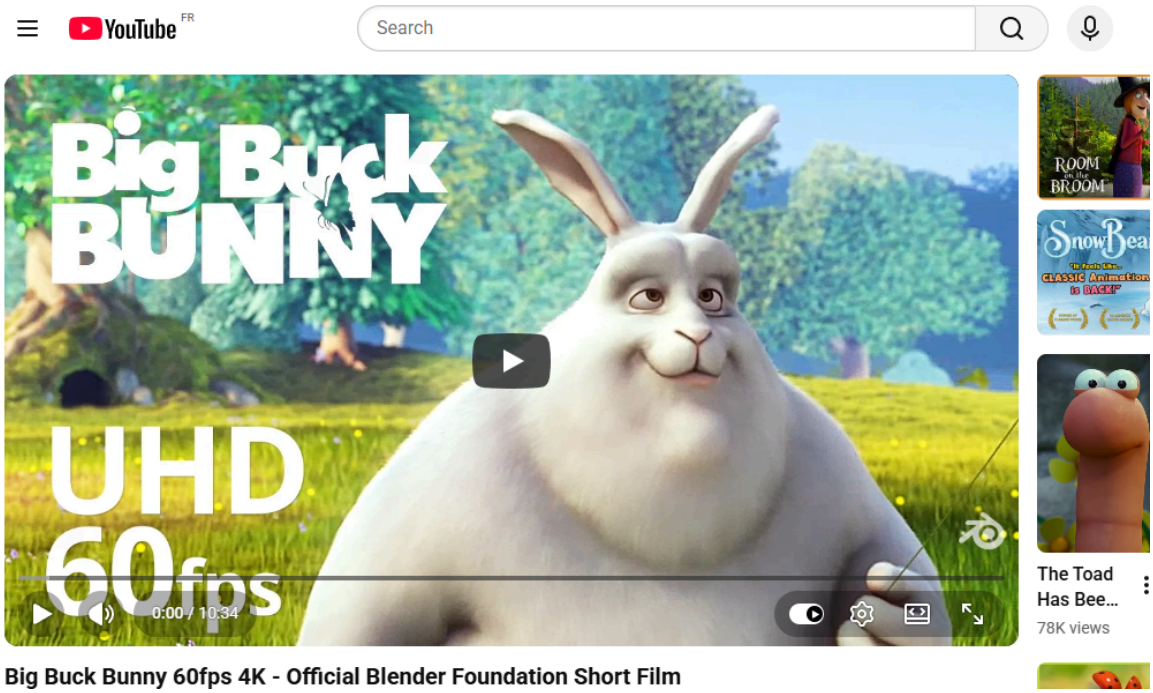
- Chemin (path) : **/blog/article/search**

- Séparateur de query string : le caractère **?**

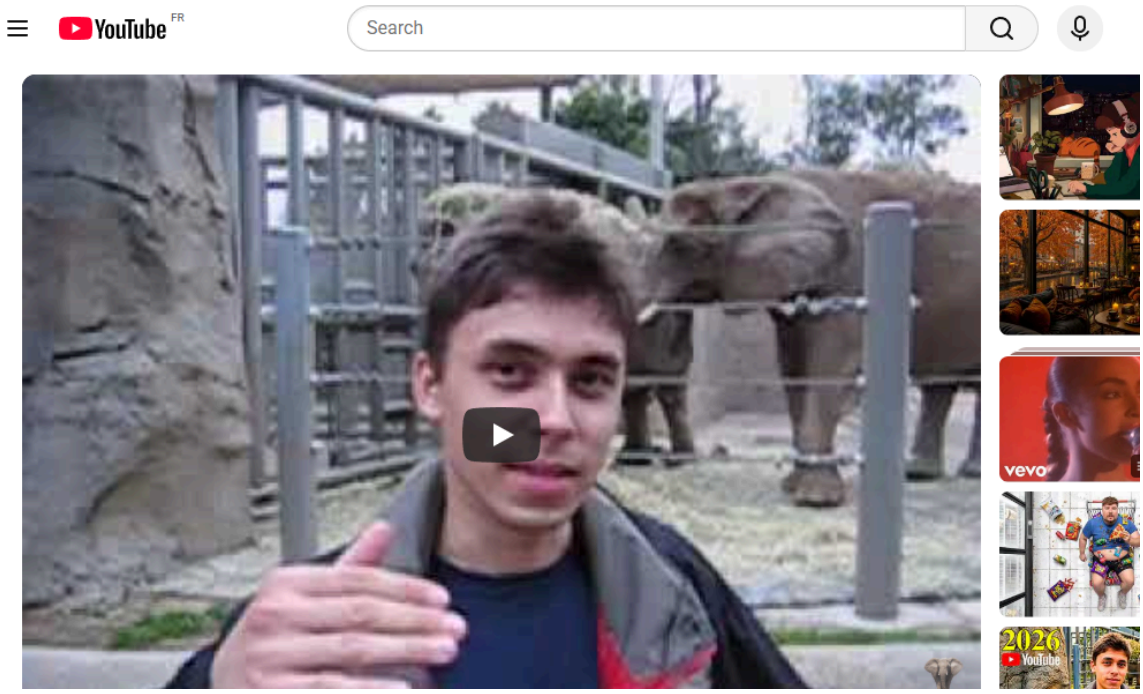
- Query string : **color=blue** et **sort=newest**, séparés par le symbole **&**

Exercice 2 : Manipulation d'un paramètre sur YouTube

On ouvre une première vidéo. Son URL est **youtube.com/watch?v=aqz-KE-bpKQ** :



Dans la barre d'adresse, on remplace seulement la valeur du paramètre **v=** par **jNQXAC9IVRw**, puis on valide :



Le serveur renvoie une **vidéo totalement différente**. On en conclut que la simple modification manuelle d'un paramètre d'URL ordonne au serveur de charger un autre contenu.

Exercice 3 : Modification de la Query String

On étudie la query string **?color=blue&sort=newest** envoyée à un site e-commerce.

La variable **color** filtre les produits par couleur et la variable **sort** définit l'ordre d'affichage. Ici, on demande les produits bleus triés du plus récent au plus ancien.

Pour obtenir les articles rouges, on réécrit la query string ainsi : **?color=red&sort=newest**.

On peut ajouter un troisième paramètre en respectant le séparateur **&**, par exemple une taille : **?color=red&sort=newest&size=large**.

Question de réflexion : comme on peut altérer à la main ces paires clé-valeur, on peut aussi tester si le serveur contrôle bien nos droits. C'est la première étape d'un audit de **contrôle d'accès**, qui mène aux attaques **IDOR**.

La faille IDOR

Le principe de la faille

IDOR signifie **Insecure Direct Object Reference** (référence directe non sécurisée à un objet).

Pour l'imaginer, on prend un placard à deux sections : celle du haut pour **John**, celle du bas pour **Mary**. Si John ouvre la section de Mary et lit ses livres sans que rien ne l'en empêche, c'est une action non sécurisée.

Sur une application web, ces “placards” sont des **objets** référencés par des **identifiants numérotés** dans l'URL. En changeant l'identifiant, un utilisateur atteint les ressources d'un autre.

L'exploitation par manipulation de paramètre

On imagine être connecté à une application de gestion avec le compte employé dont l'**identifiant utilisateur** est le numéro **11**. En consultant notre historique, l'URL affiche **/activity?user_id=11**.

L'attaque IDOR consiste à modifier cet identifiant dans la barre d'adresse :

- Test 1 : on remplace **user_id=11** par **user_id=1** (le compte du Président).

- Test 2 : on remplace par **user_id=2** (le compte de la Vice-Présidente).

Si l'application est vulnérable, le serveur affiche directement les **données confidentielles** de ces comptes.

La remédiation côté serveur

La faille vient d'un **défaut de contrôle d'autorisation** : le serveur vérifie bien qu'on est **authentifié**, mais il oublie de vérifier qu'on a le droit d'accéder à l'objet demandé.

La règle d'or : **ne jamais faire confiance aux paramètres d'URL**. Le serveur doit comparer l'identifiant demandé avec celui stocké dans la session de l'utilisateur connecté.

Le code vulnérable fait une confiance aveugle à la valeur passée dans l'URL :

```
$user_id = $_GET["user_id"];
$query = "SELECT * FROM activity WHERE user_id = " . $user_id;
```

Le code sécurisé récupère l'identifiant depuis la session vérifiée et utilise une requête préparée :

```
session_start();
$logged_user = $_SESSION["user_id"];
$stmt = $pdo->prepare("SELECT * FROM activity WHERE user_id = ?");
$stmt->execute([$logged_user]);
```

Deux cas réels sur HackerOne

On regarde deux failles IDOR réelles, signalées sur la plateforme de bug bounty **HackerOne** :

- **API d'Acronis** : l'identifiant était un token placé dans le chemin de l'URL. En le modifiant, un utilisateur accédait aux informations d'un autre. Prime : **100 $**.

- **Suppression de notifications** : en changeant le paramètre **user_id**, l'attaquant pouvait effacer les notifications d'un autre utilisateur. Prime : **500 $**.

On retient qu'une IDOR ne se limite pas à la lecture : elle peut aussi **supprimer ou modifier** des ressources. Et peu importe le format (paramètre **?user_id=** ou token dans le chemin), le problème reste le même : le serveur oublie de contrôler l'autorisation.

La divulgation de fichiers sensibles

Le fichier robots.txt

Le fichier **robots.txt**, placé à la racine d'un site, indique aux moteurs de recherche les parties à ne pas indexer, avec des règles **Allow** et **Disallow**.

Le paradoxe : en inscrivant un dossier confidentiel en **Disallow**, l'administrateur signale indirectement aux attaquants qu'il contient des ressources sensibles. Le fichier devient une carte vers les zones à explorer.

Exercice 4 : Analyse de fichiers robots.txt

On affiche le robots.txt de Wikipedia à l'adresse **en.wikipedia.org/robots.txt** :

```
# robots.txt for http://www.wikipedia.org/ and friends
#
# Please note: There are a lot of pages on this site, and there are
# some misbehaved spiders out there that go _way_ too fast. If you're
# irresponsible, your access to the site may be blocked.
#

# Observed spamming large amounts of https://en.wikipedia.org/?curid=NNNNNN
# and ignoring 429 ratelimit responses, claims to respect robots:
# http://mj12bot.com/
User-agent: MJ12bot
Disallow: /

# advertising-related bots:
User-agent: Mediapartners-Google*
Disallow: /

# Wikipedia work bots:
User-agent: IsraBot
Disallow:

User-agent: Orthogaffe
Disallow:

# Crawlers that are kind enough to obey, but which we'd rather not have
# unless they're feeding search engines.
User-agent: UbiCrawler
Disallow: /

User-agent: DOC
Disallow: /

User-agent: Zao
Disallow: /
```

On compare deux blocs. Pour **User-agent: DOC** suivi de **Disallow: /**, tout le site est interdit à ce robot. Pour **User-agent: IsraBot** suivi de **Disallow:** (vide), rien n'est interdit : le robot a le droit de tout explorer.

On affiche ensuite le robots.txt de **semrush.com** :

```
User-agent: 008
Disallow: /

User-agent: SiteAuditBot
Crawl-delay: 1
Allow: /

User-agent: Semrushbot-SI
Allow: /

User-agent: Yahoo Pipes 2.0
Disallow: /

User-agent: Facebot
Disallow: /archive/graphs.php
Allow: /academy/certificates/*

User-agent: facebookexternalhit
Allow: /academy/certificates/*

User-agent: Bingbot
Disallow: /archive/graphs.php

User-agent: BingPreview
Disallow: /archive/graphs.php

User-agent: LinkedInBot
Allow: /academy/certificates/*

User-agent: Twitterbot
Allow: /blog/*
Allow: /ranking-factors/*
Allow: /academy/certificates/*

User-agent: *
Disallow: /blog/*?img=*
Disallow: /archive/graphs.php
Disallow: /forrester-marketing-silos/*
Disallow: /partner/
Disallow: /blog/search/*
Disallow: /analytics/seomagic/
Disallow: /my_reports/reports/*
Disallow: /archive/
Disallow: /admin/
Disallow: /users/
Disallow: /clients/
Disallow: /custom_report/
Disallow: /limit_hits/
Disallow: /payment/
Disallow: /no_cookies/
Disallow: /double_activation/
Disallow: /projects/?*
Disallow: /?*/
Disallow: /activation_successful*
Disallow: /api.html?*
Disallow: /partners/
Disallow: /partner/
Disallow: */dvd-mode/
Disallow: /unsubscribe/
Disallow: /unsubscribe/*
Disallow: /demonstration/
Disallow: /demonstration/*
Allow: /projects/
Disallow: */custom-report/
Disallow: */request-demo/
Allow: */info/kdt/
Disallow: */pleasereg/
```

Le dossier **/projects/** est globalement **autorisé** à l'exploration : c'est la ligne **Allow: /projects/** qui le montre.

La ligne **Disallow: /projects/?*** interdit, elle, toutes les URL de **/projects/** qui contiennent une query string (le **?** suivi de n'importe quoi). On bloque donc les pages paramétrées ou dynamiques, tout en laissant les pages **/projects/** normales accessibles.

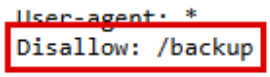
Les endpoints et fichiers de cartographie

Un **endpoint** est une URL précise sur un serveur. En plus de **robots.txt**, d'autres fichiers standard renseignent sur un site : **sitemap.xml** (la carte des pages), mais aussi des fichiers oubliés comme **.env** ou **/cgi-bin/phpinfo.php**.

On retient que **robots.txt ne sécurise rien** : il donne des consignes d'indexation mais n'empêche personne d'ouvrir l'URL directement. Les environnements de développement et de production doivent être strictement séparés, et aucun secret ne doit rester accessible.

Exercice 5 : Lab “Source code disclosure via backup files”

On résout un premier lab PortSwigger. On commence par consulter le **/robots.txt** du site, qui révèle un dossier caché avec **Disallow: /backup** :



On ouvre ce dossier **/backup**. Il contient un fichier de sauvegarde **ProductTemplate.java.bak** :



On ouvre ce fichier **.bak**. Le code source fuité contient le **mot de passe de la base de données en clair** :

```
            "postgresql",
            "localhost",
            5432,
            "postgres",
            "postgres",
            '99m1ie2n6u7qw8q0nalh347vmyktohko'
).withAutoCommit();
try
{
    Connection connect = connectionBuilder.connect(30);
```

On soumet ce mot de passe comme solution. Le lab est validé :



Exercice 6 : Lab “Information disclosure on debug page”

On résout un second lab, sur le même principe des **endpoints connus**. En testant **/cgi-bin/phpinfo.php**, on tombe sur la page de configuration PHP laissée par un développeur.

En cherchant le terme **secret** dans la page, on trouve la **SECRET_KEY** de l'application exposée en clair :

REMOTE_HOST	90.73.32.244
USER	carlos
HTTP_SEC_CH_UA	"Not:A-Brand";v="8", "Chromium";v="156", "Google Chrome";v="156"
SECRET_KEY	t2v9r7wzui0uovflyekdyfziq6t9k9mm
HTTP_SEC_FETCH_USER	?1
QUERY_STRING	*no value*

On soumet cette clé comme solution. Le lab est validé :



L'injection HTML

Le principe

L'**injection HTML** survient quand une application réaffiche directement une saisie de l'utilisateur sans la **nettoyer** (sanitization). Notre texte devient alors une partie du **code HTML** de la page.

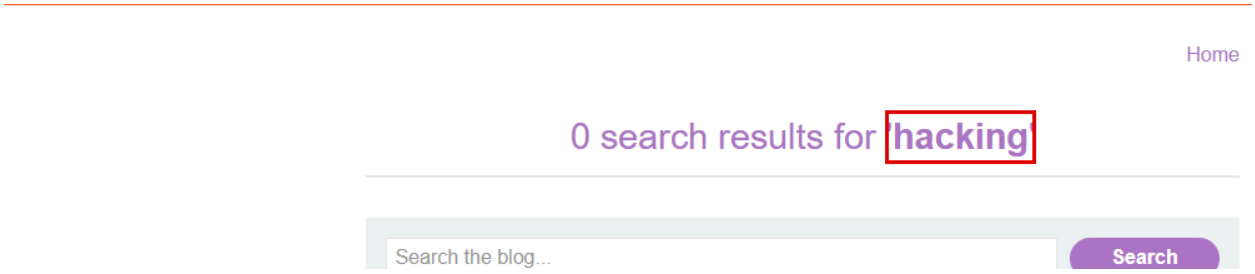
En profitant de ce défaut, un attaquant peut insérer de **faux formulaires de connexion**, ajouter des **liens trompeurs** ou **défigurer** la page. Et si l'injection contient une balise **<script>**, on passe au **XSS**, bien plus dangereux.

Une page web s'appuie sur trois langages : le **HTML** (la structure, avec des balises comme **<h1>**, **<p>** ou **<img>**), le **CSS** (le style) et le **JavaScript** (la logique).

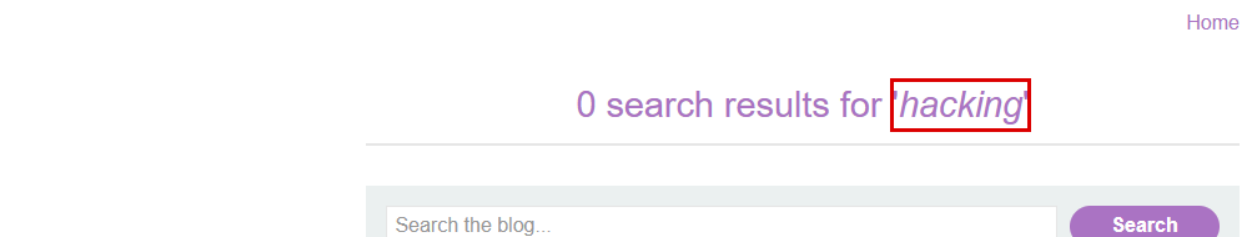
Exercice 7 : Injection de balises de mise en forme

On teste si une zone de saisie interprète les balises HTML, en utilisant le moteur de recherche d'un **blog volontairement vulnérable**.

En recherchant le mot **hacking** entouré des balises **** et ****, le mot s'affiche **en gras** : la balise a été interprétée, l'injection HTML fonctionne :



Avec les balises **** et ****, le mot s'affiche en italique :



On en déduit que le serveur ne nettoie pas la saisie : il renvoie nos balises telles quelles dans le HTML de la page.

Exercice 8 : Injection d'un faux lien

L'injection ne se limite pas à la mise en forme. Avec la balise d'ancrage **<a>**, on peut insérer un **faux lien** (fake link) qui pointe vers un site malveillant, par exemple de hameçonnage. On injecte par exemple :

```
Cher client, cliquez sur ce <a href="https://www.arnaque.com">lien</a>
```

Le texte affiche un lien d'apparence légitime, mais qui renvoie vers un site d'arnaque. On peut copier l'URL obtenue dans la barre d'adresse et l'envoyer à une victime : comme le paramètre est réaffiché sans contrôle, le lien piégé s'exécute chez elle.

La cause racine est toujours la même : l'**absence d'assainissement** des données côté serveur. La correction consiste à **neutraliser** les caractères **<** et **>** (les convertir en entités HTML) pour que la saisie soit traitée comme du texte, jamais comme du code.

Le Cross-Site Scripting (XSS)

Le principe : XSS réfléchi et XSS stocké

Le **XSS** (Cross-Site Scripting) ressemble à l'injection HTML, mais la balise injectée contient du **JavaScript exécutable** qui tourne dans le navigateur de la victime. L'impact est bien plus grave : **détournement de session**, **vol d'identifiants**, prise de contrôle du compte.

La charge utile (**payload**) classique pour prouver la faille est **<script>alert(1)</script>**.

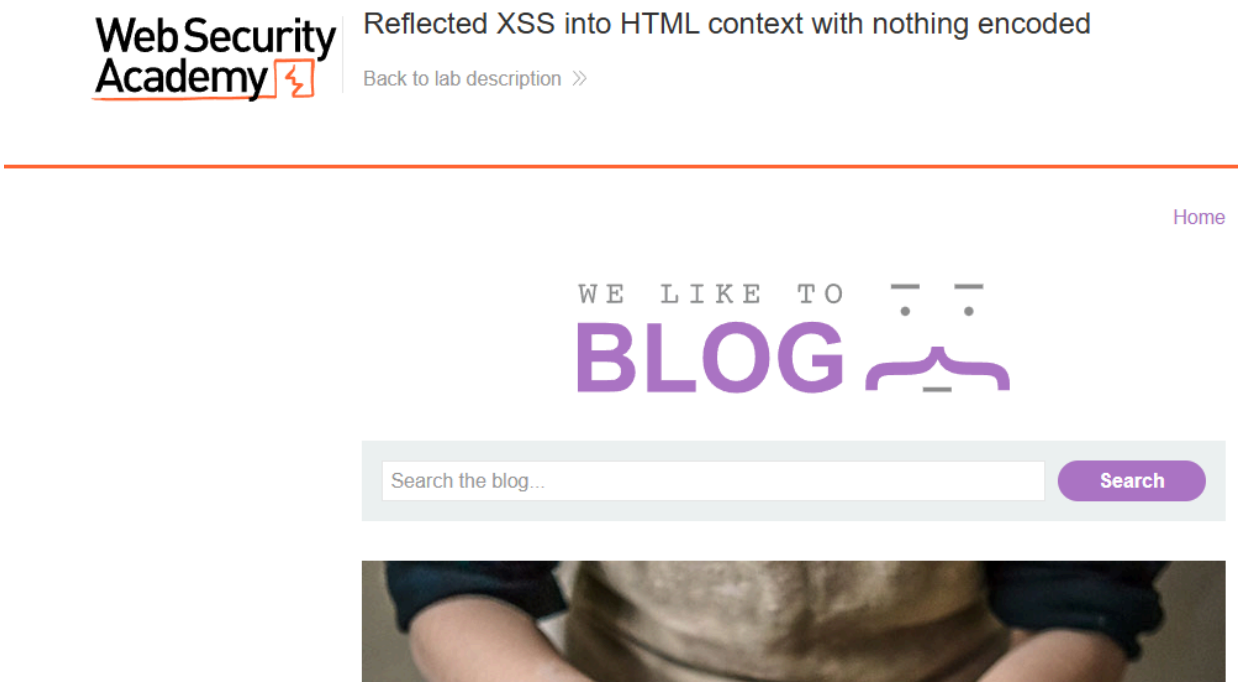
On distingue deux types :

- **XSS réfléchi** : le script n'est pas sauvegardé. L'attaquant doit transmettre un **lien piégé** que la victime clique.

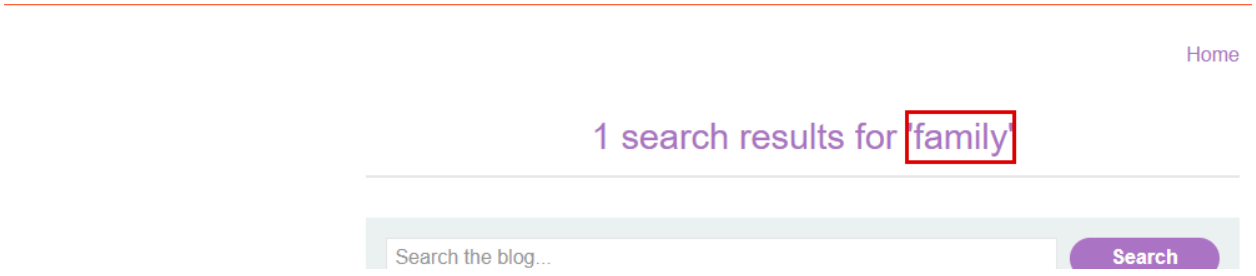
- **XSS stocké** : le script est **enregistré** sur le serveur (par exemple dans un commentaire). Il s'exécute **automatiquement** pour chaque visiteur de la page.

Exercice 9 : XSS réfléchi (lab PortSwigger)

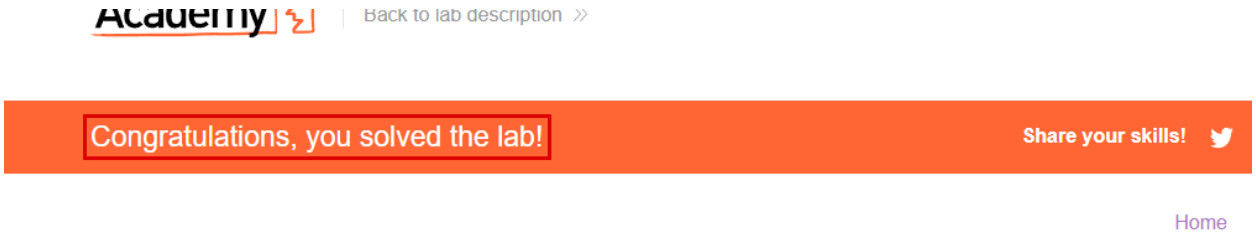
On ouvre un lab : un blog avec un moteur de recherche qui réaffiche le terme saisi dans la page :



On vérifie d'abord le comportement normal, par exemple en cherchant le mot **family** :



Comme les balises sont interprétées (vu à l'exercice 7), on injecte directement la charge utile **<script>alert(1)</script>** dans la recherche. Le JavaScript s'exécute dans le navigateur et le lab est validé :



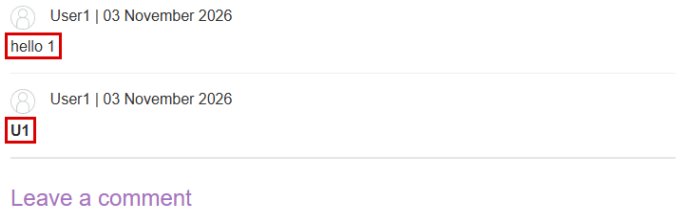
C'est bien du **XSS réfléchi** : il suffit d'envoyer à une victime le lien de recherche contenant le script pour qu'il s'exécute chez elle.

Exercice 10 : XSS stocké (lab PortSwigger)

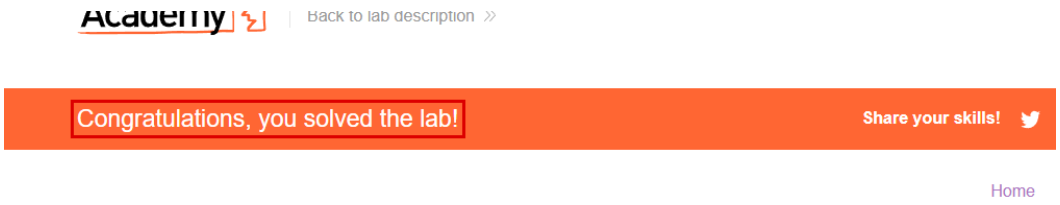
On ouvre un second lab : un blog avec une **zone de commentaires**. Tout commentaire publié est enregistré et réaffiché à chaque visite :



On publie d'abord un commentaire normal (**hello 1**), puis un commentaire contenant **U1**. Le **U1** s'affiche en gras : les balises sont interprétées, le champ ne filtre rien :



On publie alors un commentaire contenant la charge utile **<script>alert(1)</script>**. Le script est stocké et s'exécute automatiquement au chargement de la page : le lab est validé :



Le **XSS stocké** est le plus critique : une seule injection réussie touche **tous les visiteurs** de la page, sans aucun clic sur un lien externe.

L'injection SQL

Les bases du langage SQL

Le **SQL** (Structured Query Language) est le langage que les sites et les applications utilisent pour **communiquer avec une base de données** : extraire (**SELECT**), ajouter (**INSERT**), modifier (**UPDATE**) ou supprimer (**DELETE**) des données.

Une requête de sélection se décompose en trois parties. Par exemple **SELECT * FROM users WHERE status = 'active'** :

- **SELECT *** : les colonnes à récupérer (ici, toutes).

- **FROM users** : la table où chercher l'information.

- **WHERE status** = **'active'** : le filtre, la condition à respecter.

Chaque connexion ou recherche sur un site déclenche une requête SQL en arrière-plan. Si cette requête est mal construite, elle devient vulnérable à l'**injection SQL**, l'une des failles les plus critiques du web.

La mécanique de l'injection SQL

Lors d'une connexion, le serveur vérifie l'identifiant avec une requête du type **SELECT * FROM customers WHERE first_name = 'John' AND last_name = 'Doe'**, où le prénom joue le rôle de l'identifiant et le nom celui du mot de passe.

Pour contourner cette vérification **sans connaître le mot de passe**, l'attaquant détourne la requête avec trois caractères réservés :

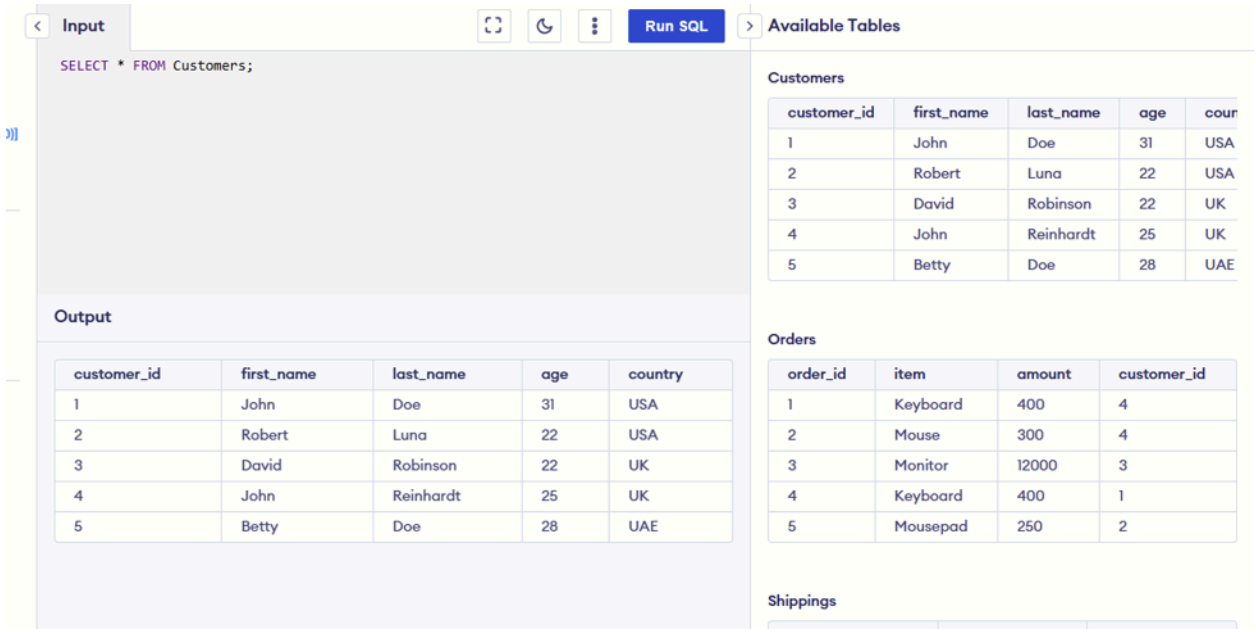
- L'apostrophe ' : elle ferme la chaîne de caractères par anticipation et permet de « sortir » du texte attendu.

- Le double tiret -- : il met en commentaire tout ce qui suit, donc la vérification du mot de passe est **ignorée** par le serveur.

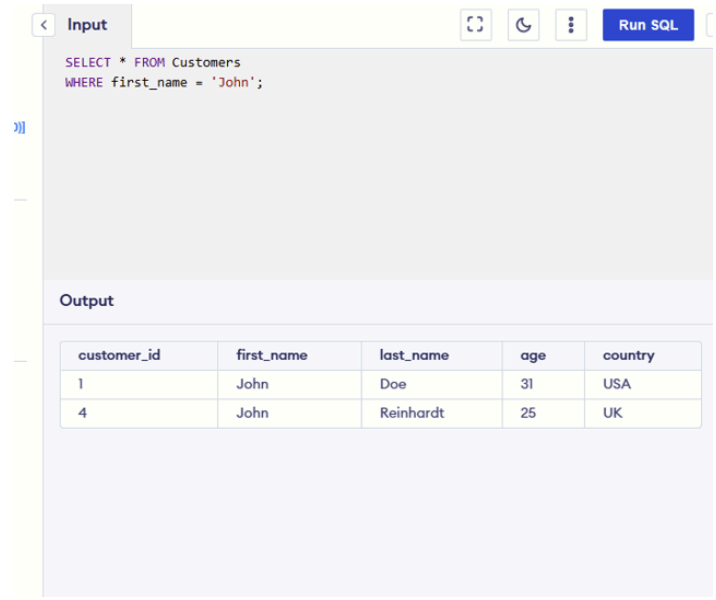
- La charge ' **OR 1=1 --** : **1=1** est toujours vrai, donc la condition globale devient vraie et le serveur valide la connexion comme si le mot de passe était correct.

Exercice 11 : Manipulation de requêtes sur un compilateur SQL

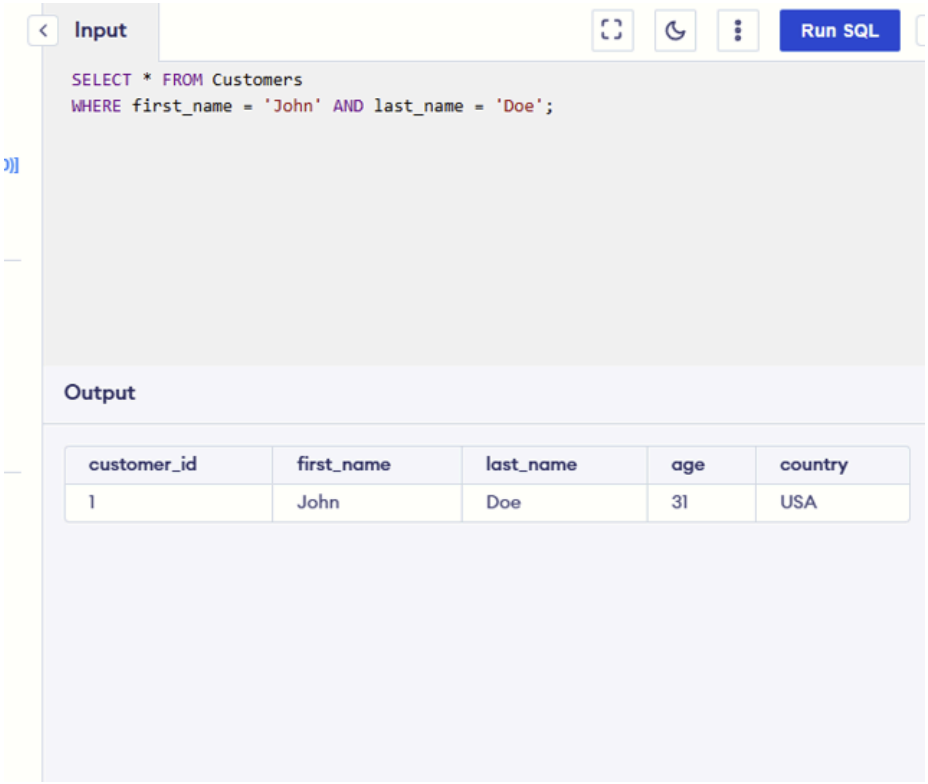
On utilise le compilateur SQL en ligne **Programiz** (programiz.com/sql/online-compiler). La base d'exemple contient **3 tables** : **Customers**, **Orders** et **Shippings**. On affiche d'abord tous les clients avec **SELECT * FROM Customers** :



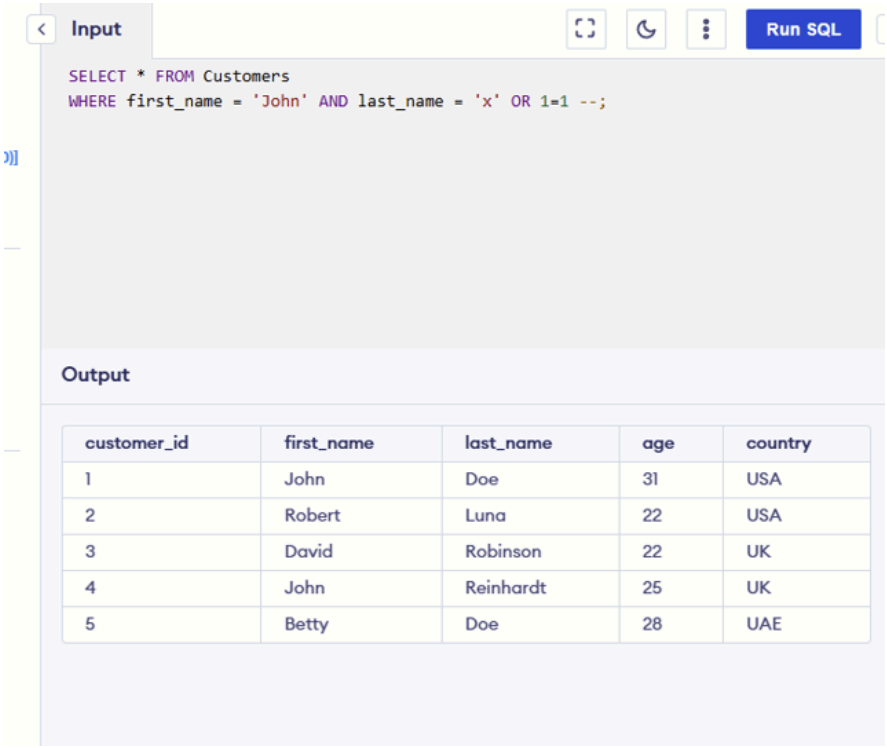
On filtre les clients dont le prénom est John avec **WHERE first_name = 'John'**. La requête renvoie **2 occurrences** (John Doe et John Reinhardt) :



On affine avec le prénom **John** ET le nom **Doe** (**AND last_name = 'Doe'**). Il ne reste plus qu'**1 occurrence** : c'est exactement le principe d'une vérification de connexion (identifiant + mot de passe) :



On démontre enfin l'injection. Même avec un mot de passe faux (**last_name = 'x'**), en ajoutant **OR 1=1 --**, la condition devient toujours vraie et la base renvoie **toutes les lignes** : la vérification est contournée.



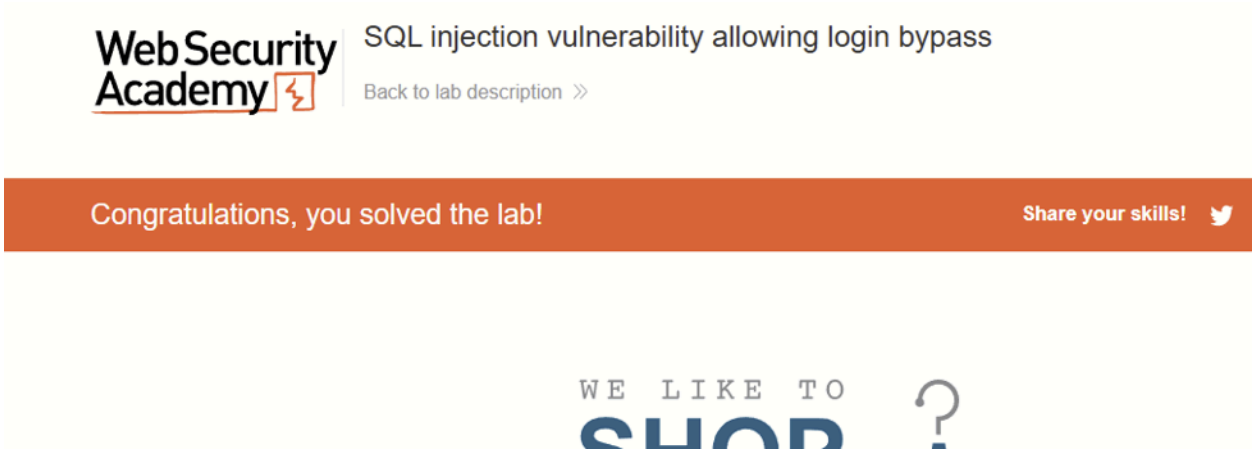
Exercice 12 : Lab “SQL injection vulnerability allowing login bypass”

On résout un lab PortSwigger : un blog dont la **fonction de connexion** est vulnérable. L'objectif est de se connecter en tant qu'**administrator** sans connaître son mot de passe.

Sur la page de connexion, on saisit dans le champ **Username** la charge utile **administrator'--** et n'importe quel mot de passe :



La requête exécutée devient **SELECT * FROM users WHERE username = 'administrator'--' AND password = '...'**. L'apostrophe ferme le nom d'utilisateur et le **--** met en commentaire toute la vérification du mot de passe. On est alors connecté en **administrator** et le lab est validé :



Conclusion

Ce TP a montré qu'on peut pratiquer la sécurité web **sans rien installer**, avec un simple navigateur.

On a vu plusieurs familles de failles : le **contrôle d'accès** (IDOR), la **divulgation de fichiers sensibles**, l'**injection HTML**, le **XSS** et l'**injection SQL**. Toutes s'exploitent depuis le navigateur et viennent du même défaut : le serveur fait confiance à des données qu'il ne contrôle pas.

Les cinq labs **PortSwigger** ont été résolus. On retient les bons réflexes de défense : ne jamais faire confiance au client, **assainir** toute saisie réaffichée, utiliser des **requêtes préparées** pour la base de données, séparer la production du développement et ne jamais exposer un secret sur une page publique.